- **Purpose of Homework:**
  - Algorithm design and analysis, like any skill, can only be developed through consistent practice and feedback. Whether it's cooking, playing basketball, integration, gardening, interviewing, or teaching, theoretical knowledge alone is not sufficient. The comfortable feeling of "Oh, sure, I get it" after following a well-presented lecture or hearing a TA explain a homework solution is a seductive, yet dangerous trap. True understanding comes from doing the thing—by actually solving the problems yourself.
  - The homework assignments are your opportunity to practice. Lectures, textbooks, office hours, labs, and guided problem sets are designed to build intuition and provide justification for the skills we want you to develop. However, the most effective way to develop those skills is by attempting to solve the problems on your own. The process is far more important than the final solution.
  - Expect to get stuck. It's normal to have no idea where to start on some problems. That's why you have access to a textbook, lecture slides, and discussions. The journey of wrestling with the problem is an essential part of the learning process.

Homework Guidelines

- **Collaboration and Academic Integrity:**
  - You are encouraged to work together on homework problems, but you must list everyone you worked with for each problem.
  - You must code everything based on your interpretation and properly cite every external source you use, including ideas from other students. The only sources that you are not required to cite are the official course materials (lectures, notes, homework solutions).
  - Plagiarism is strictly prohibited. Using ideas from other sources or people without citation is considered plagiarism. Copying verbatim from any source, even with citation or permission, is also considered plagiarism. Don't cheat.
- **Programming Guidelines:**
  - You are encouraged, but not required, to write clean and descriptive code. This helps with debugging, especially if you want to ask for help at office hours.
  - All solutions must be coded from scratch, i.e. we will **not** provide boilerplate code for any programming problem. One implication is that you must write your own code to read the inputs and print the outputs based on our convention.
  - Pay close attention to the instructions for each problem. Your code might fail some test cases due to non-algorithimic errors, e.g. integer overflow, runtime errors, stack overflow, etc.

Policy on Academic Integrity and AI Usage

- **LLM Usage:** To ensure the development of independent algorithmic thinking, the use of Large Language Models (e.g., ChatGPT, Gemini) for generating assignment solutions is strictly discouraged.
- **Oral Verification:** At the conclusion of the semester, the instructor reserves the right to perform *randomized oral examinations* on submitted work.
  - Students must be able to provide a precise, step-by-step explanation of their logic and the underlying mechanics of their algorithm.
  - Failure to demonstrate a thorough understanding of the submitted work during this oral defense will result in a rejection of the assignment.

Note: Authenticity in your reasoning is the primary criterion for evaluation.

Programming Assignment #1 Guidelines

Points

- Programming Assignment #1 is worth up to **250 points**.
- Students enrolled in the **Honors** option are required to solve all programming problems.
- **Honors points count for everyone:** Everyone else is welcome to attempt them as well. If you solve any Honors problems, the points you earn are added to your total homework points in the same way as the rest of the assignment.

Overview

Problem	File Name	Time Limit	Points
Effortless Scheduling	`scheduling.*`	1 second	20
Bouncers	`bouncers.*`	1 second	30
Long Walk	`walk.*`	1 second	30
Birthday Present	`present.*`	1 second	50
Kingdom Surveys	`surveys.*`	1 second	50
Chocolate Giveaway	`giveaway.*`	1 second	70
			Total: 250

The `.*` corresponds to the file extension of the programming language of your choice. Note that the time limits above are the **base rate** only, the actual time limit will be multiplied with the factors below depending on the language.

Language	C++	C	Java	Python	Rust
Extension	`.cpp`	`.c`	`.java`	`.py`	`.rs`
Time Multiplier	1x	1x	2x	5x	2.5x

For example, if you submit a Python solution for Effortless Scheduling, your program must run within 5×1 second $= 5$ seconds for every test case.

Submission Instructions:

- Submit your solutions on Gradescope, one file per problem. Each submitted file **must** follow this format: `[problem name].*`. Failure to adhere to this format will result in your solution not being graded.
- The following languages are supported: **C++, C, Java, Python, Rust**. However, you are **NOT** restricted to these languages only; let us know your favorite language and we will create a grading pipeline for it :)
- For each language, you are only allowed to use standard libraries as part of your code. Usage of external libraries (e.g. `numpy`, `sortedcontainers` in Python) will result in a **Compilation Error**.
- **Local Testing:** We also provide a ZIP file containing the sample tests and a Makefile for each supported language. To test your solution locally, extract our ZIP file, move the Makefile for your language of choice and write your source code in the main directory, then run `make run PROBLEM=<problem name>` to verify your solution against the sample tests. Feel free to add your own test cases.

1 Effortless Scheduling

Tan, Harry, and Brandon are peer mentors for CS577. At office hours, they face a line of n students who want to ask questions. Per course policy, each student is allowed to ask one question, which takes the peer mentors one minute to answer.

However, not all questions are easy: some students just want to know when the midterm is, while others are asking for a proof of $P \neq NP$! Fortunately, Harry just built a new AI model that predicts the difficulty of each question. For the i-th student, the AI predicts their question will require a_i effort; note that a_i can be negative as some questions are purely logistical!

To keep things orderly, each mentor also has their own rules for scheduling office hours.

- Tan wants office hours to be disjoint, i.e., at any given moment, only one peer mentor is active.
- Brandon wants office hours to be contiguous. He wants to wake up, go to Morgridge Hall, answer questions, and then go home and sleep for the rest of the day. He is too lazy to go back to Morgridge Hall.
- Harry wants to ensure that a student doesn't have to attend two different office hours. If a peer mentor starts answering a student's question, then he will finish answering that question.

After a 6-second deliberation, Tan, Harry, and Brandon agreed on the following order: Harry will hold h minutes of office hours, followed by t minutes with Tan, and finally by b minutes with Brandon. Each mentor has to actually do work, meaning $h, t, b \geq 1$. Unsurprisingly, all peer mentors have their own work to do, so no one wants to spend more effort than anyone else.

Subject to all these rules, how many ways are there to distribute the times h, t, and b so that the total effort that each mentor expends is the same?

Input

The first line of input contains an integer n, the total number of students.

The second line of input contains n integers $a_1, \ldots, a_n$, the effort it takes to answer each student's questions.

Output

Output the number of ways to evenly split Tan, Harry, and Brandon's office hours.

Constraints

- $|a_i| \leq 10^9$
- $1 \leq n \leq 10^6$

Examples

Sample Input	Sample Output
5 1 2 3 0 3	2

Sample Input	Sample Output
4 0 1 -1 0	1

Sample Input	Sample Output
2 4 1	0

Explanation

In the first test case, the possible distributions are $(2, 1, 2)$ and $(2, 2, 1)$.

In the second test case, the only possible distribution is $(1, 2, 1)$.

In the third test case, there is no fair distribution. Office hours are cancelled :(

2 Bouncers

Marceline the Vampire Queen is hosting a concert at the Hamburger Hills Cemetery to promote her new singles! However, after a previous incident caused by her dad last concert, she wants to avoid concert crashers and asks Peppermint Butler to hire m bouncers to prevent this. Marceline's concert will last n hours long, and each bouncer i is available for help from hour ℓ_i to hour r_i (inclusive).

Unfortunately, her electric bass breaks at the last minute, so Marceline has to spend money to repair it! While her bass is now functional, she only has enough money left to pay all but two bouncers! She still wants to maximize the amount of time her concert is guarded by at least one bouncer. Can you help her select the appropriate bouncers?

Input

The first line of input contains two integers n, m, the concert length and the number of available bouncers.

m lines of input follow. The ith line contains two integers ℓ_i, r_i, the availability of bouncer i.

Output

Output a single number, the maximum bouncer availability if Marceline dismisses the two bouncers optimally.

Constraints

- $1 \leq n \leq 10^6$, $3 \leq m \leq 5000$
- $1 \leq \ell_i \leq r_i \leq n$ for all $1 \leq i \leq m$.

Examples

Sample Input	Sample Output
7 5 1 4 4 5 5 6 6 7 3 5	7

Explanation

In the sample test case, Marceline should dismiss bouncer 2 and 5, then the remaining bouncers can still cover all 7 hours of her concert.

3 Long Walk

One day, in the freezing winters at UW-Madison, Bucky Badger finds himself at home eating junk food. However, he starts to feel guilty about all the weight he has gained, so he wants to go for a walk. But he really, really doesn't want to be outside.

Strangely, the UW-Madison campus lies on the number line. There are n heated buildings, and the ith building spans the interval $[a_i, b_i]$. To have the feeling that he is making progress, Bucky plans a one-directional walk, i.e., he either walks from the left side of campus to the right or from the right side of campus to the left. He is free to choose anywhere on campus to start his walk, including the inside of buildings.

Bucky walks at a pace of 1 unit per minute, and he's willing to tolerate k minutes in the freezing cold. To maximize exercise, he wants to plan the longest walk possible. How long can Bucky make his walk? You may ignore any work Bucky did to get to his starting point, along with any work needed for him to go home.

Input

The first line of input contains two integers n, k, the number of heated buildings on campus, and the amount of freezing time that Bucky can tolerate.

n lines of input follows. The i-th line contains two integers a_i, b_i, indicating that there is a building that spans the interval $[a_i, b_i]$.

Output

Output the maximum amount of distance that Bucky can walk if he choose an optimal starting point.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$, $1 \leq k \leq 10^9$
- For each building i, $1 \leq a_i \leq b_i \leq n$. It is also guaranteed that the buildings do not overlap, and that they are listed from left to right, i.e. $a_1 \leq b_1 < a_2 \leq b_2 < \cdots < a_n \leq b_n$.

Examples

Sample Input	Sample Output
3 4 2 5 7 9 10 11	10

Sample Input	Sample Output
5 10 5 7 11 12 16 20 25 26 30 33	18

Explanation

In the first sample test, if Bucky starts his walk at position 2, he can exercise for 10 minutes before surrendering to the cold. No other starting points can give him a longer exercise.

4 Birthday Present

Finn and Jake are preparing a birthday present for BMO! They just found an old but colorful piece of cloth hidden underneath the treasure room, and Finn wants to give the cloth directly to BMO. Jake proposes instead to use the cloth as a frame by cutting out some piece in the middle and putting the trio's picture there.

BMO is specially programmed to see the world as a grid of RGB colors; he sees the cloth as an $m \times n$ grid of integers $c_{i,j}$ where each integer is one out of k colors programmed in his brain. The **beauty** of the cloth is the number of distinct colors BMO can see. In addition, the trio's picture can be interpreted as an $h \times w$ sub-rectangle over the grid. Note that the picture is **strictly** smaller in size compared to the cloth.

Jake wants to cut out some portion of the cloth while still ensuring the newly cut cloth is as beautifully as possible. Unfortunately, he is very indecisive about the cut position, so he asks Finn to find the cloth's beauty when cutting at **all** possible $h \times w$ subpieces. More formally, for every $1 \leq x \leq m - h + 1$ and $1 \leq y \leq n - w + 1$, Jake wants an answer to the following question:

- How many distinct colors remain on the cloth if every color $c_{i,j}$ with $x \leq i \leq x + h - 1$ and $y \leq y \leq c + w - 1$ is removed from the cloth?

Despite Finn's frequent usage of mathematical terms, he is not excellent in mathematics himself, so he asks for your help to give BMO the best ever birthday present!

Input

The first line of input contains five integers m, n, k, h, w, the cloth dimensions, the number of colors BMO can see and the picture dimensions, respectively.

m lines of input follow, the ith line contains c integers $c_{i,1}, \ldots, c_{i,n}$ describing the colors of the cloth.

Output

Output $m-h+1$ lines where each line contains $n-w+1$ integers. For line x, the yth number denotes the cloth's beauty if the rectangular subpiece with upper left corner (x, y) and bottom right corner $(x+h-1, y+w-1)$ is cut out.

Constraints

- $m, n, k \leq 10^6$. In addition, $mnk \leq 10^6$.
- $1 \leq w \leq m, 1 \leq h \leq n$. In addition, $(w, h) \neq (m, n)$.
- $1 \leq c_{i,j} \leq k$ for all $1 \leq i \leq m$, $1 \leq j \leq n$.

Examples

Sample Input	Sample Output
3 4 5 2 2 2 2 1 1 3 2 5 3 3 4 4 3	4 4 3 5 3 4

Explanation

In the sample test, the cloth and all possible 2×2 cutouts for the picture are depicted as follows:

2	2	1	1
3	2	5	3
3	4	4	3

Original cloth

		1	1
		5	3
3	4	4	3

Cut out $(1, 1)$

2			1
3			3
3	4	4	3

Cut out $(1, 2)$

2	2		
3	2		
3	4	4	3

Cut out $(1, 3)$

2	2	1	1
		5	3
		4	3

Cut out $(2, 1)$

2	2	1	1
3			3
3			3

Cut out $(2, 2)$

2	2	1	1
3	2		
3	4		

Cut out $(2, 3)$

5 Kingdom Surveys

The streets of Candy Kingdom have degraded significantly, owing in part to many recent attacks and weird phenomena in the kingdom recently. Princess Bubblegum has decided to reconstruct the roads again, and she is planning this project with Peppermint Butler.

Peppermint Butler has just drawn a map of the Candy Kingdom, which can be represented as an $m \times n$ binary grid whose upper left border is $(1, 1)$ and lower right border is (m, n). Each cell on the grid is either a `0` (denoting an empty plot) or a `1` (denoting a house). Since the candy citizens often got lost in the kingdom, Princess Bubblegum zoned the kingdom so that there is **at most one path** (going through only houses) between any two houses.

Peppermint Butler has also written down a list of neighborhoods in the Candy Kingdom; here, a *neighborhood* is a subgrid of houses such that any two houses are connected through a sequence of adjacent houses. After seeing the map and the list, Princess Bubblegum has come up with q questions to survey the kingdom. Each of her question has the following form:

- How many neighborhoods are formed by the houses residing in the rectangular subregion with upper-left corner $(x_1^{(i)}, y_1^{(i)})$ and lower-right corner $(x_2^{(i)}, y_2^{(i)})$?

Although Peppermint Butler can answer these questions promptly, he prefers to move on to other more important (avnd and interesting) tasks of the project. Can you help answer them in his place?

Input

The first line of input contains two integers m, n, the dimensions of the Candy Kingdom.

m lines of input then follow. The ith line contains a binary string s_i of length n, where $s_{i,j} = \texttt{1}$ indicates a house at (i, j) and `0` indicates an empty plot there.

The next line of input contains an integer q, the number of questions Princess Bubblegum will ask.

q lines of input then follow. The ith line contains four integers $x_1^{(i)}, y_1^{(i)}, x_2^{(i)}, y_2^{(i)}$, denoting a question for the subregion whose upper-left corner is $(x_1^{(i)}, y_1^{(i)})$ and lower-right corner is $(x_2^{(i)}, y_2^{(i)})$.

Output

For each query, output a single line containing the answer to the corresponding query.

Constraints

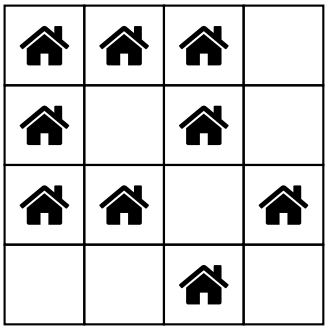
- $1 \le m, n \le 10^6$. In addition, $1 \le m \times n \le 10^6$.
- $1 \le q \le 10^6$.
- For each query i, $1 \le x_1^{(i)} \le x_2^{(i)} \le m$ and $1 \le y_1^{(i)} \le y_2^{(i)} \le n$.

Examples

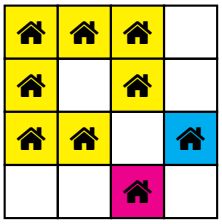
Sample Input	Sample Output
4 4 1110 1010 1101 0010 4 1 1 4 4 1 1 3 3 1 2 3 2 3 1 4 4	3 1 2 3

Explanation

In the sample test, the map of the Candy Kingdom and the subregion for each query is depicted as follows:



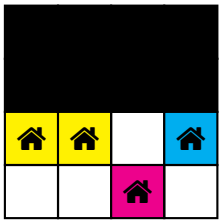
Original map



Query $(1,1) \to (4,4)$

Query $(1,1) \to (3,3)$

Query $(1,2) \to (3,2)$



Query $(3,1) \to (4,4)$

6 Chocolate Giveaway

Professor Manolis has decided to hand out free chocolates from Dubai! For the next q days, he will host one handout event per day, located in one of the dorms around campus.

- The UW-Madison campus contains n dorms. There are $n-1$ roads that connect these dorms, and there exists a path that connect any two given dorms.
- On day $i \in [q]$, an event will be hosted at dorm u_i. Any student that attends event i will receive x_i Dubai chocolates. To attend an event, students must walk from their dorm to the event and back.
- Unfortunately, there is construction on campus. Specifically, on day i, there is construction around dorm v_i. When there is construction around a dorm, student can neither enter nor leave. Moreover, a student cannot walk a path if there is construction around any dorm on that path. The construction people are mean, so they will always hold construction in a dorm that is right next to Professor Manolis's event.

The natural question is as follows. For each dorm $j \in [n]$, determine how many Dubai chocolates can be obtained by a student that lives at dorm j.

Input

The first line of input contains an integer n, the number of dorms on campus.

The next $n-1$ lines describe the roads between dorms. The jth line contains two integers a_j, b_j, indicating that there is a road between dorm a_j and dorm b_j. It is guaranteed that $1 \leq a_j, b_j \leq n$, and that $a_j \neq b_j$.

The next line of input contains an integer q, the total number of days. q lines of input follow. The ith line contains three integers u_i, v_i, x_i, indicating that the event on day i will be held at u_i, that Professor Manolis will give x_i chocolates to each student, and that there will be construction around dorm v_i.

Output

Output a single line containing n numbers, where the i-th number is the total number of chocolates that can be obtained by a student living in dorm i.

Constraints

- $2 \leq n \leq 5 \cdot 10^5$.
- For each road, it is guaranteed that $1 \leq a_j, b_j \leq n$, and that $a_j \neq b_j$.
- $1 \leq q \leq 5 \cdot 10^5$.
- For each event, it is guaranteed that $1 \leq u_i, v_i \leq n$, and that $|x_i| \leq 10^9$. Moreover, u_j and v_j are always connected by a road.

Examples

Sample Input	Sample Output
5 1 2 2 3 2 4 4 5 4 1 2 1 4 5 2 2 1 3 3 2 4	3 5 9 5 3